dlprim · catalog of operators

Every method in catalog.py, grouped. A **group** is an abstract operator: a base class whose `__call__` says what all its methods do ("Convert"). A **method** is a concrete operator: a frozen dataclass that subclasses its group with the type parameters filled in ("RandomFlip"). Its fields are its settings, and calling it runs it.

> **Expressions** work on one batch in memory and change nothing.
>
> **Statements** make new state (create), change state in place (update, the argument marked `!`, returns None), or read a whole dataset (dataset). A statement is always a line of its own, never inside an expression.
>
> **R** is what else a method takes (nothing by default): `Best(Fold[Path, Path, [Value]])` takes a Path and a Value and gives a Path. Pyright checks every method's signature against its group's.

211 concrete operators (methods) in 31 groups, 33 abstract operators (base classes) · 122 expressions, 89 statements (20 change their argument) · 172 model-agnostic, 39 model-dependent

The abstract operators

family	group (base class)	what its methods take and give	methods
EXPRESSIONS			
structure	`Convert[A, B, **R = []]`	`(x: A, /, *R) -> B`	8
	`Map[A, B, **R = []]`	`(x: A, /, *R) -> B`	9
	`Aggregate[C, **R = []]`	`(x: C, /, *R) -> C`	4
	`Select[C, **R = []]`	`(x: C, /, *R) -> C`	2
	`Fold[C, A, **R = []]`	`(x: C, /, *R) -> A`	7

	`Scan[X, S, Y]`	`(xs: Seq, /) -> Seq`
		`init() -> S`
		`step(state: S, x: X, /) -> tuple[S, Y]`
			7

learning	`Split[C, **R = []]`	`(x: C, /, *R) -> Example`	18
	`Score[P, O = Tensor, **R = []]`	`(pred: P, /, *R) -> O`	26
partial	`Start[P, **R = []]`	`(*R) -> P`	3
	`Fill`	`Holes[C] → C`	1
	`Order`	`(holes: Holes[Any], pred: Prediction, /) -> Holes[Any]`	4
	`Walk`	`PartialTree → Path`	2
	`Choose`	`(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]`	6
	`Unroll`	`PartialSeq[S] → S`	2
	`Evolve[S]`	`(state: S, /) -> S`	2
	`Reshape[P, **R = []]`	`(p: P, /, *R) -> P`	13
lookup	`Search[O, **R = []]`	`(index: Index, /, *R) -> O`	3

models	`Wrap[M, **R = []]`	`(model: M, /, *R) -> nn.Module`	5

STATEMENTS

create	`MapLayer[A, B, **R = []]`	`(*R) -> Model[A, B]`	17
	`AggregateLayer[C, In = C, **R = []]`	`(*R) -> Model[In, C]`	10
	`FoldLayer[C, A, **R = []]`	`(*R) -> Model[C, A]`	1
	`Encode[A, B, **R = []]`	`(*R) -> Codec[A, B]`	3
	`Empty[S, **R = []]`	`(*R) -> S`	2
	`Copy[M]`	`(x: M, /) -> M`	1
	`Sklearn[E: Estimator]`	`Estimator → SkModel`	9
update	`Fit[X, **R = []]`	`(x: X, /, *R) -> None`	6
	`Assign[X, **R = []]`	`(x: X, /, *R) -> None`	3
	`Alter[X, **R = []]`	`(x: X, /, *R) -> None`	7
	`Store[X, **R = []]`	`(x: X, /, *R) -> None`	4
dataset	`Source[D, **R = []]`	`(*R) -> D`	4
	`Curate[C, **R = []]`	`(data: Indexed[C], /, *R) -> Subset[C]` `indices(data: Indexed[C], /, *R) -> Tensor`	7

	`Partition[C, **R = []]`	`(data: Indexed[C], /, *R) ->`
		`tuple[tuple[Subset[C], ...], ...]`
		`parts(data: Indexed[C], /, *R) ->`
		`tuple[tuple[Tensor, ...], ...]`
			3

	`Reduce[C, A, **R = []]`	`(data: Dataset[C], /, *R) -> A`	12

EXPRESSIONS · on a batch, nothing changes

STRUCTURE · data → data

`Convert[A, B, **R = []]` C[a] → C'[a]

`(x: A, /, *R) -> B`

Change the container, keep what the elements are.

`Patchify(Convert[Grid, Grid])` Patching agnostic

`(x: Grid, /) -> Grid`

SETTINGS `size: int`

An image → a coarser grid of p×p patches, each one element (a reshape); a linear map after it is a patch embedding.

`Flatten(Convert[Grid, Seq])` Flattening agnostic

`(x: Grid, /) -> Seq`

A grid → a sequence in raster order: the lattice is forgotten, learned positions say where each element was.

`Concat(Convert[Seq, Seq])` Multimodal sequence agnostic

`(*parts: Seq) -> Seq`

Several sequences of the same width as one (image patches, then caption tokens): one network reads them all.

`RandomFlip(Convert[Grid, Grid])` Data augmentation: random flip agnostic

`(x: Grid, /) -> Grid`

SETTINGS `p: float = 0.5`

Mirror each image with probability p: its elements move, none changes.

`RandomCrop(Convert[Grid, Grid])` Data augmentation: random crop agnostic

`(x: Grid, /) -> Grid`

SETTINGS `padding: int = 2`

Pad, and cut back at a random offset: a random translation.

`Transitions(Convert[Path, Value])` Experience (transitions) agnostic

`(path: Path, returns: Seq | None = None, /) -> Value`

The steps of walked paths as a set of transitions (state, action, reward, next state, done, and the action's log-probability), their order dropped: what a replay buffer holds and RL losses read.

`PathTo(Convert[PartialTree[Any, Any], Path, [Tensor]])` A path from a tree new agnostic

`(tree: PartialTree[Any, Any], leaf: Tensor, /) -> Path`

The steps from the root to a leaf of a tree's known part: the result of a walk.

`InContextLearning(Convert[Seq, Seq])` In-context learning (few-shot prompts) agnostic

→ Concat

Solved examples, then the query, as one prompt: the model adapts by reading them.

`Map[A, B, **R = []]` C[a] → C[b]

```
(x: A, /, *R) -> B
```

Change each element on its own, keep the container.

`RoPE(Map[Tensor, Tensor, [Tensor]])` Positional encoding: rotary (RoPE) agnostic

```
(x: Tensor, positions: Tensor, /) -> Tensor
```

Rotate pairs of query and key features by angles proportional to the position, so that attention depends only on relative offsets.

`VectorQuantize(Map[Container, Container, [Tensor]])` Vector quantization agnostic

```
(x: Container, codebook: Tensor, /) -> Container
```

Each element → the id of its nearest codebook entry (real → classes), with a straight-through gradient.

`StraightThrough(Map[Tensor, Tensor, [Tensor]])` Straight-through estimator agnostic

```
(hard: Tensor, soft: Tensor, /) -> Tensor
```

Forward the hard value, backpropagate through the soft one: a gradient for a choice that has none.

`Condition(Map[Container, Container, [Value]])` Conditioning agnostic

```
(x: Container, label: Value, /) -> Container
```

Attach a label to every element (its one-hot vector next to each element's values; a none label for guidance).

`ColorJitter(Map[Grid, Grid])` Data augmentation: color jitter agnostic

```
(x: Grid, /) -> Grid
SETTINGS strength: float = 0.2
```

Scale each image's brightness and contrast by random factors.

`GaussianNoise(Map[Container, Container])` Data augmentation: additive noise agnostic

```
(x: Container, /) -> Container
SETTINGS sigma: float = 0.05
```

x + σ·ε at every element.

`LabelSmoothing(Map[Value, Value])` Label smoothing agnostic

```
(y: Value, /) -> Value
SETTINGS eps: float = 0.1
```

A class → a distribution over classes, (1 − ε)·one_hot + ε / n: never pushed to infinite confidence.

`Attribution(Map[Container, Container, [Predictor]])` Attribution (input × gradient) agnostic

```
(x: Container, model: Predictor, /) -> Container
```

How much each input element pushed the output: the input times the output's gradient, on any differentiable model.

`Chinchilla(Map[float, tuple[float, float]])` Compute-optimal allocation (Chinchilla) agnostic

```
(compute: float, /) -> tuple[float, float]
```

A compute budget → (parameters, tokens): C ≈ 6·N·D with D ≈ 20 tokens per parameter.

`Aggregate[C, **R = []]` C[h] → C[h]

```
(x: C, /, *R) -> C
```

Each element reads the elements it is related to: a C in, a C out.

`Mixup(Aggregate[tuple[Container, Value]])` Data augmentation: mixup — agnostic

```
(pair: tuple[Container, Value], /) -> tuple[Container, Value]
```

SETTINGS `alpha: float = 0.2`

Each example blended with a random partner of the batch, labels by the same λ: the batch as a Set, paired at random.

`Cutmix(Aggregate[tuple[Grid, Value]])` Data augmentation: cutmix — agnostic

```
(pair: tuple[Grid, Value], /) -> tuple[Grid, Value]
```

A random box pasted from a partner image; the labels mix by its area.

`BatchBaseline(Aggregate[Value])` Baseline: the batch mean — agnostic

```
(returns: Value, /) -> Value
```

Each return minus the batch's mean: an advantage with less variance and no bias.

`GroupRelative(Aggregate[Value])` GRPO (group-relative advantages) — agnostic

```
(rewards: Value, /) -> Value
```

SETTINGS `n: int`

Each reward standardized within its group of n samples from one prompt: an advantage without a critic.

`Select[C, **R = []]` C[a] → C[a]

```
(x: C, /, *R) -> C
```

Keep, drop, reorder or reweight elements: a C in, a C out.

`QualityFilter(Select[Container, [Value]])` Data filtering — agnostic

```
[C: Container](x: C, scores: Value, /) -> C
```

SETTINGS `threshold: float`

Keep the examples of a batch scoring above a threshold (a classifier's, a verifier's, a perplexity). Keeping the best fraction needs every batch: KeepBest.

`TokenDropout(Select[Seq])` Data augmentation: token dropout — agnostic

```
(x: Seq, /) -> Seq
```

SETTINGS `p: float = 0.1`

Drop a random fraction of the positions; the rest keep their order.

`Fold[C, A, **R = []]` C[a] → a

```
(x: C, /, *R) -> A
```

Reduce a container C to one value A.

`DecayGroups(Fold[nn.Module, list[dict[str, Any]]])` Weight decay (parameter groups) — agnostic

```
(model: nn.Module, /) -> list[dict[str, Any]]
```

SETTINGS `weight_decay: float = 0.1`

The parameters to decay (the weight matrices) apart from the rest (norms, biases, embeddings).

`MuPGroups(Fold[nn.Module, list[dict[str, Any]]])` Parameterization (μP) agnostic

```
(model: nn.Module, /) -> list[dict[str, Any]]
```

SETTINGS `width: int, base_width: int`

Hidden matrices reweighted by base_width / width, so a learning rate tuned narrow carries over wide.

`DiscountedReturn(Fold[Path, Value])` Expected return (the RL objective) agnostic

```
(path: Path, /) -> Value
```

SETTINGS `gamma: float = 1.0`

Σ γ^t·r_t along each path: what RL maximizes. A walk can't be differentiated through, so its gradient comes from scoring the path (reinforce).

`Best(Fold[Path, Path, [Value]])` Best-of-N with a verifier agnostic

```
(paths: Path, scores: Value, /) -> Path
```

SETTINGS `n: int`

The best-scoring of each group of n paths (scored by a verifier or a reward model).

`Majority(Fold[Value, Value])` Self-consistency (majority vote) agnostic

```
(answers: Value, /) -> Value
```

SETTINGS `n: int`

The most frequent answer of each group of n samples.

`EarlyStop(Fold[Sequence[Tensor], bool])` Early stopping agnostic

```
(curve: Sequence[Tensor], /) -> bool
```

SETTINGS `patience: int = 3`

Whether the validation scores, lowest best, have not improved on their best for `patience` evaluations.

`FitPowerLaw(Fold[Seq, tuple[float, float], [Seq]])` Scaling laws (power-law fit) agnostic

```
(sizes: Seq, losses: Seq, /) -> tuple[float, float]
```

loss ≈ a·size^(−b), fitted on the logs.

`Scan[X, S, Y]` Seq[X] → Seq[Y]

```
(xs: Seq, /) -> Seq
init() -> S
step(state: S, x: X, /) -> tuple[S, Y]
```

Carry a state along a sequence: init gives the first state, step takes a state and an element to the next state and an output; calling it runs the steps along a sequence (from the end when reverse).

`Optimizer(Scan[Tensor, Tensor, Tensor])` Optimizers (SGD, momentum, Adam, Muon) new agnostic

```
init() -> Tensor
step(state: Tensor, grad: Tensor, /) -> tuple[Tensor, Tensor]
```

SETTINGS `kind: str = "adamw", lr: float = 0.0003`

STATE the moments of every parameter

Gradients in, updates out: SGD carries nothing, momentum a velocity, Adam two moments, Muon orthogonalizes. A setting of Train, which carries its state from step to step.

`WarmupCosine(Scan[int, int, float])` Learning-rate schedules agnostic

```
init() -> int
step(state: int, t: int, /) -> tuple[int, float]
```

SETTINGS `warmup: int, total: int`

STATE the step count

A learning rate for each step: linear warmup, then cosine decay. A setting of Train.

`Lagrangian(Scan[tuple[Tensor, Tensor], float, Tensor])` Constrained objectives (Lagrangian) agnostic

```
init() -> float
step(state: float, losses: tuple[Tensor, Tensor], /) -> tuple[float, Tensor]
```

SETTINGS `lr: float = 0.1`

STATE the multiplier λ ≥ 0

(loss f, constraint g) in, f + λ·(g − c) out; λ rises while the constraint is violated (dual ascent).

`ReturnsToGo(Scan[Tensor, Tensor, Tensor])` Returns-to-go agnostic

```
init() -> Tensor
step(state: Tensor, reward: Tensor, /) -> tuple[Tensor, Tensor]
```

SETTINGS `gamma: float = 1.0`

STATE the return from here on

G_t = r_t + γ·G_{t+1}: what each step led to, scanned from the end.

`Negamax(Scan[Tensor, Tensor, Tensor])` Self-play agnostic

```
init() -> Tensor
step(state: Tensor, reward: Tensor, /) -> tuple[Tensor, Tensor]
```

STATE the outcome for the player to move

One policy plays both sides; each move's return is the outcome for whoever made it: G_t = r_t − G_{t+1}.

`GAE(Scan[tuple[Tensor, Tensor], Tensor, Tensor])` Value functions, Bellman, GAE agnostic

```
init() -> Tensor
step(state: Tensor, reward_value: tuple[Tensor, Tensor], /) -> tuple[Tensor, Tensor]
```

SETTINGS `gamma: float = 0.99, lmbda: float = 0.95`

STATE the advantage from here on

A critic's TD errors r + γV(s′) − V(s), averaged backward with weights (γλ)^k; each element a (reward, value).

`ParallelScan(Scan[Tensor, Tensor, Tensor])` Parallel scan agnostic

→ torch.cumsum

s_t = a_t·s_{t−1} + b_t for every t at once, in logarithmic depth: how a recurrence trains in parallel.

LEARNING · data → a training signal

`Split[C, **R = []]` C → Example

```
(x: C, /, *R) -> Example
```

Hide part of a container C; what was hidden (or a teacher's answer) is the target.

`SplitFor(Split[Container])` Choosing the split from the data agnostic

```
(x: Container, /) -> Example
```

The split a container's type picks: classes in order → Shift · other classes → Mask · real values → Noise.

`Labelled(Split[Container, [Container]])` Supervised new agnostic

```
(x: Container, y: Container, /) -> Example
```

The data comes split: the input with nothing hidden, the label as the target.

`Classification(Split[Container, [Container]])` Classification new agnostic

→ Labelled, Loss

Supervised learning with classes as the target: Labelled, then Loss picks cross-entropy (a categorical's negative log-likelihood); Accuracy measures it. Any model that predicts the classes does it: Linear (logistic regression), Network, KNN, a boosted-trees estimator.

`Regression(Split[Container, [Container]])` Regression `new` agnostic

→ Labelled, Loss

Supervised learning with real values as the target: Labelled, then Loss picks squared error (a Gaussian's negative log-likelihood). Linear with LeastSquares is linear regression; Network, KNN and boosted trees do it too.

`Shift(Split[Seq])` Self-supervised: next-token prediction agnostic

```
(x: Seq, /) -> Example
```

Each element is predicted from the ones before it (a causal Seq): the target at i is the element at i + 1.

`Mask(Split[Container])` Self-supervised: masked prediction (BERT, MAE, MaskGIT) agnostic

```
(x: Container, /) -> Example
```

SETTINGS `ratio: float | None = None`

Random elements at level 1 ([MASK] for classes, 0 for real values): 15% is BERT, 75% MAE, a random ratio per example MaskGIT and masked diffusion.

`Noise(Split[Container])` Diffusion / score matching / flow matching agnostic

```
(x: Container, /) -> Example
```

Every element at one random level t ($x_t = (1 - t)\cdot x + t\cdot\varepsilon$): the target is the velocity, the noise or the clean data, rescalings of one another.

`NoiseEach(Split[Seq])` Diffusion forcing `new` agnostic

```
(x: Seq, /) -> Example
```

A random level for each element: next-token prediction and diffusion are its two extremes.

`NextFrame(Split[Product])` Self-supervised: future-frame prediction agnostic

```
(x: Product, /) -> Example
```

The frames up to t as the input, the frames from t + 1 as the target: shift, on the frames of a video.

`Reconstruction(Split[Container])` Reconstruction (autoencoder) agnostic

```
(x: Container, /) -> Example
```

Nothing hidden; the target is the input itself, so the model's bottleneck is what makes it hard.

`TwoViews(Split[Container, [Callable[[Container], Container]]])` Contrastive views agnostic

```
(x: Container, augment: Callable[[Container], Container], /) -> Example
```

Two random augmentations of the same examples, each the other's target (CLIP pairs an image with its caption).

`Distillation(Split[Container, [Predictor]])` Knowledge distillation agnostic

```
(x: Container, teacher: Predictor, /) -> Example
```

The target is a teacher's (softened) prediction on the same input.

`SelfDistillation(Split[Container, [Predictor, Callable[[Container], Container]]])` Self-distillation (BYOL / DINO / JEPA) agnostic

```
(x: Container, teacher: Predictor, augment: Callable[[Container], Container], /) -> Example
```

One view as the input; the target is a slow (EMA) copy's embedding of another view: agree with an older self.

`SFT(Split[Seq])` Instruction tuning / SFT agnostic

```
(x: Seq, /) -> Example
```

SETTINGS `prompt_len: int`

shift, with only the response as targets: the prompt is read, never predicted.

`TDTarget(Split[Value, [Predictor]])` TD learning (value regression) agnostic

`(batch: Value, critic: Predictor, /) -> Example`

SETTINGS `gamma: float = 0.99`

V(s) learns toward r + γ·V(s′): a target bootstrapped from the critic's own estimate.

`QTarget(Split[Value, [Predictor]])` Q-learning (DQN) agnostic

`(batch: Value, q: Predictor, /) -> Example`

SETTINGS `gamma: float = 0.99`

Q(s, a) learns toward r + γ·max Q(s′, ·) at the action taken (the weight picks it out).

`Dynamics(Split[Value])` World-model targets (dynamics) agnostic

`(batch: Value, /) -> Example`

From each transition, the input is the state and the action; the target what followed (next state, reward, done). A model of it is a learned follow (imagined).

`SearchTargets(Split[Path, [PartialTree[Any, Any]]])` Amortized search (AlphaZero targets) agnostic

`(path: Path, search: PartialTree[Any, Any], /) -> Example`

For each state a walk visited: what search made of it (its visit distribution) and what followed (the return).

`Score[P, O = Tensor, **R = []]` Prediction × C → ℝ

`(pred: P, /, *R) -> O`

How bad a prediction P is (against R: a target, a weight): every loss, every surrogate objective, every metric. O is a number to minimize (a Tensor), or one per example (a Value).

`Objective(Score[nn.Module, Tensor, ...])` Objective (a split, then its loss) new agnostic

`(model: nn.Module, /, *batch: Any) -> Tensor`

SETTINGS `split: Split[Any, ...]`

The loss of a model on a batch: split it into an Example, predict from the input, score with the loss the target picks (labelled data: split = Labelled()).

`Loss(Score[Prediction, Tensor, [Container]])` Loss = negative log-likelihood of the target agnostic

`(pred: Prediction, target: Container, weight: Tensor | None = None, /) -> Tensor`

−log p(target), chosen by what the target holds: classes → cross-entropy · bits → BCE · real → squared error · distribution → KL · record → the sum of its fields' losses.

`CrossEntropy(Score[Tensor, Tensor, [Container]])` Cross-entropy agnostic

`(logits: Tensor, target: Container, weight: Tensor | None = None, /) -> Tensor`

Classes, or a distribution over them (soft labels).

`BinaryCrossEntropy(Score[Tensor, Tensor, [Container]])` Binary cross-entropy agnostic

`(logits: Tensor, target: Container, weight: Tensor | None = None, /) -> Tensor`

Bits.

`Hinge(Score[Tensor, Tensor, [Value]])` Hinge loss (SVM) new agnostic

`(scores: Tensor, target: Value, weight: Tensor | None = None, /) -> Tensor`

SETTINGS `margin: float = 1.0`

max(0, margin − y·f(x)) for labels ±1 (for classes: the margin over the best wrong class): a support vector machine is a Linear trained with it (on KernelFeatures for a kernel SVM).

`SquaredError(Score[Tensor, Tensor, [Container]])` Squared error agnostic

`(pred: Tensor, target: Container, weight: Tensor | None = None, /) -> Tensor`

Real values: a unit Gaussian's negative log-likelihood, up to a constant.

KL(Score[Dist, Tensor, [Dist]]) KL divergence agnostic

```
(pred: Dist, target: Dist, weight: Tensor | None = None, /) -> Tensor
```

A distribution against a reference (a prior, a reference policy).

PerField(Score[Prediction, Tensor, [Container]]) Multi-task training agnostic

```
(pred: Prediction, target: Container, weight: Tensor | None = None, /) -> Tensor
```

A record: one loss per field, each chosen by what the field holds, summed.

LogProb(Score[Prediction, Value, [Container]]) Log-probability new agnostic

```
(pred: Prediction, x: Container, /) -> Value
```

log p(x) for each element under a prediction, unreduced: what losses and estimators are built from.

PathLogProb(Score[PartialTree[Any, Any], Value, [Path]]) Teacher forcing (the log-probability of a path) agnostic

```
(tree: PartialTree[Any, Any], path: Path, /) -> Value
```

Σ log options(node)(edge) along a path; on the data's own path, −1/length of it is the next-token loss.

SequenceLogProb(Score[Predictor, Value, [Seq]]) Sequence log-probability agnostic

```
(model: Predictor, x: Seq, /) -> Value
SETTINGS start: int = 1
```

log p(x[start:] | the rest before it), per sequence: teacher forcing on whole sequences, in one pass.

InfoNCE(Score[Value, Tensor, [Value]]) Contrastive (InfoNCE / CLIP) agnostic

```
(a: Value, b: Value, /) -> Tensor
SETTINGS temperature: float = 0.07
```

Which b goes with each a, among the batch: cross-entropy over similarities, the matching pair as the class.

BradleyTerry(Score[Value, Tensor, [Value]]) Reward models (Bradley–Terry) agnostic

```
(chosen: Value, rejected: Value, /) -> Tensor
```

−log σ(r(chosen) − r(rejected)): the likelihood that the preferred answer wins.

DPOLoss(Score[nn.Module, Tensor, ...]) Preferences (DPO) agnostic

```
(model: nn.Module, /, *batch: Any) -> Tensor
SETTINGS reference: Predictor, beta: float = 0.1
```

The loss of (policy, chosen, rejected): Bradley–Terry on β·(the policy's log-ratio to the reference, chosen minus rejected). Preferences as classification: no reward model, no walk.

KLRegularizedReward(Score[Value, Value, [Value, Value]]) KL-regularized reward (RLHF) agnostic

```
(reward: Value, LogProb: Value, ref_log_prob: Value, /) -> Value
SETTINGS beta: float = 0.1
```

The reward, minus how far the policy has moved from a reference: use it as the return.

ELBO(Score[Model[Container, Dist], Tensor, [Model[Tensor, Prediction], Container]]) VAE (evidence lower bound) agnostic

```
(encoder: Model[Container, Dist], decoder: Model[Tensor, Prediction], x: Container, /) -> Tensor
```

The reconstruction from a latent drawn from the posterior (rsample: reparameterization), plus KL(posterior ‖ prior).

GAN(Score[Model[Tensor, Container], tuple[Tensor, Tensor], [Predictor, Container, Tensor]])
GAN (two players) agnostic

```
(generator: Model[Tensor, Container], discriminator: Predictor, real: Container, z: Tensor, /) -> tuple[Tensor, Tensor]
```

The discriminator tells real from generated; the generator wants its samples called real. Two scores, for two players (alternate their steps).

FlowLogProb(Score[Container, Value, [list[Any]]]) Normalizing flows model-dependent

```
(x: Container, transforms: list[Any], /) -> Value
```

The exact log-likelihood through invertible maps (change of variables).

Reinforce(Score[Path, Tensor, [Value]]) Policy gradients (REINFORCE) agnostic

```
(path: Path, advantage: Value, /) -> Tensor
```

−E[log p(path)·A]: a walked path's log-probability, weighted by how much better than usual it went.

PPOClip(Score[Value, Tensor, [Value, Value]]) PPO clipped surrogate agnostic

```
(LogProb: Value, old_log_prob: Value, advantage: Value, /) -> Tensor
SETTINGS clip: float = 0.2
```

The policy gradient with the ratio π / π_old clipped: it stops pushing once the policy has moved far.

Pathwise(Score[Dist, Tensor, [Model[Tensor, Tensor]]]) Pathwise estimator (through a critic) agnostic

```
(policy: Dist, critic: Model[Tensor, Tensor], /) -> Tensor
```

−E[Q(a)] with a = rsample(): differentiated through the sample into a learned critic (DDPG, SAC).

EntropyBonus(Score[Dist]) Maximum-entropy RL (entropy bonus) agnostic

```
(options: Dist, /) -> Tensor
SETTINGS alpha: float = 0.01
```

−α·H(options): keeps the policy exploring.

MAMLLoss(Score[nn.Module, Tensor, [Callable[..., Tensor], Container, Container]]) Meta-learning (MAML) agnostic

```
(model: nn.Module, objective: Callable[..., Tensor], support: Container, query: Container, /) -> Tensor
```

Bilevel: the query loss of the weights after an inner step (an unroll) on the support set, differentiated through.

Accuracy(Score[Tensor, Tensor, [Container]]) Metrics (accuracy) agnostic

```
(pred: Tensor, target: Container, /) -> Tensor
```

Reported and selected on, never differentiated: a loss stands in for it in training.

RetrievalAccuracy(Score[Value, Tensor, [Value]]) Retrieval accuracy new agnostic

```
(a: Value, b: Value, /) -> Tensor
```

How often each a is closest to its own b among the batch (matching images to captions).

Autodiff(Score[Tensor, Tensor]) Backpropagation / autodiff agnostic
→ torch.autograd

The gradient of any score.

PARTIAL CONTAINERS · known in part, and how to get the rest

`Start[P, **R = []]` … → Partial

`(*R) -> P`

A partial container P to grow, from R: a tree for a walk, holes for a fill. (An empty index or buffer is new state: Empty, a statement.)

`Blank(Start[Holes[Container], [Container]])` Blank (where generation starts) **new** agnostic

`[C: Container](like: C, /) -> Holes[C]`

Every element unknown: all [MASK] for classes, pure noise for real values.

`LMTree(Start[PartialTree[Seq, Tensor], [Predictor, Seq]])` An LM as a tree agnostic

`(model: Predictor, prompt: Seq, /) -> PartialTree[Seq, Tensor]`

A node is a sequence (Holes: known up to its end), its options the next-token distribution, an edge appends.

`EnvTree(Start[PartialTree[Any, Tensor], [Environment]])` MDP (an environment as a tree) agnostic

`(env: Environment, policy: Predictor | None = None, /) -> PartialTree[Any, Tensor]`

A node is a state, its options the policy's actions (uniform with none), an edge leads where the environment steps to, with a reward and whether it ended.

`Fill` Holes[C] → C

fill the holes, an order deciding which and how fast: a split, undone

`Fill` Fill **new** agnostic

`[C: Container](holes: Holes[C], model: Predictor, order: Schedule, /) -> C`

SETTINGS `steps: int`

STATE Holes[C] — the levels fall, as the order decides

Repeatedly predict the holes and lower their levels as the order decides (generation, inpainting, denoising), until every level is 0. With levels in (0, 1), what the model predicts is the flow back to the data. steps is how many predictions.

`Order` Holes × Prediction → Holes

`(holes: Holes[Any], pred: Prediction, /) -> Holes[Any]`

Which holes a fill fills next, and how far: a fill's schedule, from the holes and the model's prediction.

`InOrder(Order)` Autoregressive order **new** agnostic

`(holes: Holes[Any], pred: Prediction, /) -> Holes[Any]`

STATE Holes[C] — known up to i

Fill the first hole: one element at a time, in order (the holes view of decoding; walk gives the tree view).

`MostConfident(Order)` Unmasking (MaskGIT, masked diffusion) **new** agnostic

`(holes: Holes[Any], pred: Prediction, /) -> Holes[Any]`

STATE Holes[C] — some holes at 1, the rest known

Fill the holes the model is surest of, more each step (a cosine schedule), until none is left.

`Together(Order)` Sampling a flow (diffusion) **new** agnostic

`(holes: Holes[Any], pred: Prediction, /) -> Holes[Any]`

STATE Holes[C] — every element at level t

Lower every level by one step along the predicted velocity (Euler): noise, undone.

`PerElement(Order)` Diffusion forcing sampling new agnostic

`(holes: Holes[Any], pred: Prediction, /) -> Holes[Any]`

STATE Holes[C] — a level for each element

Each element follows its own schedule (the near future clean, the far future noisy): between InOrder and together.

`Walk`

PartialTree → Path

grow the known part of a tree, a chooser deciding which leaves to keep, then read a path

`Walk` Walk new agnostic

`(tree: PartialTree[Any, Any], choose: Chooser, /) -> Path`

SETTINGS `steps: int`

STATE PartialTree — its known part, grown by the chooser

Grow the known part of a tree from its roots, steps times, a chooser deciding which edges to follow and which leaves to keep; then the path to a kept leaf. Sampling, greedy decoding, beam search, best-of-N and RL rollouts are walks with different choosers.

`ChainOfThought(Walk)` Chain-of-thought / longer reasoning agnostic

→ Walk

SETTINGS `steps: int`

STATE PartialTree — its known part, grown by the chooser

A longer walk before the answer; the gains come from training the walk (R1).

`Choose`

PartialTree → PartialTree

`(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]`

A walk's step: from the tree expanded so far, the next (which edges to follow, which leaves to keep).

`Sample(Choose)` Sampling new agnostic

`(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]`

STATE PartialTree — one leaf per root

Expand each kept leaf by one edge drawn from its options.

`Greedy(Choose)` Greedy decoding agnostic

`(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]`

STATE PartialTree — one leaf per root

Expand each kept leaf by its likeliest edge.

`Beam(Choose)` Beam search agnostic

`(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]`

SETTINGS `k: int`

STATE k leaves per root and their log-probabilities

Expand every kept leaf; keep the k likeliest children of each root.

`Independent(Choose)` Several paths per root (best-of-N, groups) new agnostic

`(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]`

SETTINGS `n: int, choose: Chooser`

STATE n independent leaves per root

n paths from each root side by side, each grown by its own chooser: n samples per prompt (best-of-N, GRPO).

`Forced(Choose)` Teacher forcing (walking the data's path) `new` agnostic

`(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]`

SETTINGS `edges: Seq`

STATE one leaf per root, on the data's path

Expand each leaf by the next given edge: scoring the path walked is teacher forcing.

`Speculative(Choose)` Speculative decoding agnostic

`(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]`

SETTINGS `draft: PartialTree[Any, Any], k: int = 4`

STATE the accepted leaf, and a draft model's guesses past it

The draft walks k edges ahead greedily, the tree's model checks them in one pass, the agreeing prefix is kept: greedy decoding, in fewer of the big model's passes.

`Unroll` PartialSeq[S] → S

step a state until done

`Unroll` Unroll agnostic

`(seq: PartialSeq[Any], /) -> list[Any]`

SETTINGS `steps: int`

STATE S — any state

The list a PartialSeq stands for: start, step(start), step(step(start)), ... steps of them.

`Iteration(Unroll)` Iteration (rounds of a program) agnostic

→ repeat

SETTINGS `steps: int`

STATE S — any state

Rounds of collecting and learning (policy iteration, self-play, expert iteration): a script's repeat (program.py), since each round's fit is a statement.

`Evolve[S]` S → S

`(state: S, /) -> S`

An unroll's step, S → S, where there is no gradient to follow or the gradient is followed with noise.

`Langevin(Evolve[Container])` Energy-based models (Langevin) agnostic

`(state: Container, /) -> Container`

SETTINGS `energy: Predictor, step_size: float = 0.01`

STATE a sample, moving

x ← x − η·∇E(x) + √(2η)·ε: samples of exp(−energy), with the gradient from autograd.

`EvolutionStrategy(Evolve[Dist])` Evolution strategies (no gradient) agnostic

`(state: Dist, /) -> Dist`

SETTINGS `fitness: Callable[[Tensor], Tensor], population: int = 32`

STATE a search distribution over the parameters

Draw a population (a Set of parameters) from the search distribution, score them, move it toward the better ones: it works when nothing is differentiable (CMA-ES keeps a covariance too).

`Reshape[P, **R = []]` Partial → Partial

```
(p: P, /, *R) -> P
```

Change how a partial container (or a model) P goes on: its options, its model. A P in, a P out.

`Temperature(Reshape[PartialTree[Any, Any]])` Sampling temperature — agnostic

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
```

SETTINGS `t: float`

The options' logits divided by t: below 1 sharper (toward greedy), above 1 flatter.

`TopK(Reshape[PartialTree[Any, Any]])` Top-k sampling — agnostic

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
```

SETTINGS `k: int`

Only the k likeliest edges stay possible.

`TopP(Reshape[PartialTree[Any, Any]])` Top-p (nucleus) sampling — agnostic

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
```

SETTINGS `p: float`

Only the likeliest edges whose probabilities add up to p stay possible.

`Given(Reshape[Predictor, [Value | None]])` A condition, fixed new — agnostic

```
(model: Predictor, condition: Value | None, /) -> Predictor
```

A conditional model with its condition fixed (None: no condition): the two models guidance mixes.

`Guided(Reshape[Predictor, [Predictor]])` Classifier-free guidance — agnostic

```
(cond: Predictor, uncond: Predictor, /) -> Predictor
```

SETTINGS `w: float`

u + w·(c − u): extrapolate from the unconditional prediction toward the conditional one, for a tree's options or a fill's model.

`EpsilonGreedy(Reshape[PartialTree[Any, Any]])` Exploration vs exploitation (ε-greedy) — agnostic

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
```

SETTINGS `eps: float`

With probability ε a uniform edge, else the likeliest.

`WithTool(Reshape[PartialTree[Any, Any], [Callable[[Any], Tensor]]])` Tool use — agnostic

```
(tree: PartialTree[Any, Any], tool: Callable[[Any], Tensor], /) -> PartialTree[Any, Any]
```

SETTINGS `call: int`

When a walk emits the call token, follow writes the tool's answer (a calculator, a search, an interpreter).

`Cached(Reshape[PartialTree[Any, Any]])` KV caching new — agnostic

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
```

STATE each known node keeps its keys and values

The tree's known nodes keep what the model computed for them, so expanding a node costs one position.

`Imagined(Reshape[PartialTree[Any, Any], [Model[Container, Container]]])` World models (a learned follow) — agnostic

```
(tree: PartialTree[Any, Any], world: Model[Container, Container], /) -> PartialTree[Any, Any]
```

The environment's follow (and reward, done) replaced by a learned model of them: walks, searches and plans in imagination (Dreamer, MuZero).

`Planned(Reshape[PartialTree[Any, Any]])` Planning (random shooting / MPC) agnostic

```
(tree: PartialTree[Any, Any], world: PartialTree[Any, Any] | None = None, /) -> PartialTree[Any, Any]
```

SETTINGS `horizon: int = 5, samples: int = 64`

STATE the random plans tried from each node

Options that look ahead: random action sequences simulated in a world, all the probability on the first action of the best.

`MCTS(Reshape[PartialTree[Any, Any]])` MCTS (tree search) agnostic

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
```

SETTINGS `simulations: int = 64, two_player: bool = False`

STATE a search tree per node: priors, visit counts, values

Options improved by search: grow a tree by PUCT (the options as priors), back up rewards and values, return the visit counts. Walk it to search; train toward it to amortize the search (AlphaZero).

`Droppable(Reshape[Predictor])` Classifier-free guidance training new agnostic

```
(model: Predictor, /) -> Predictor
```

SETTINGS `p: float = 0.1`

The condition dropped with probability p while training, so one model is both conditional and unconditional.

`kNNOptions(Reshape[PartialTree[Any, Any], [Index, Predictor]])` kNN-LM (a memory of contexts) new agnostic

```
(tree: PartialTree[Any, Any], memory: Index, embed: Predictor, /) -> PartialTree[Any, Any]
```

SETTINGS `weight: float = 0.25`

Options mixed with the next tokens of the k nearest stored contexts: an LM that can look things up.

LOOKUP · a set known by how to search it

`Search[O, **R = []]` Index × Value → Set

```
(index: Index, /, *R) -> O
```

What is built on looking up an index (index.search): the k elements nearest each query, put to use as an O.

`Retrieve(Search[Seq, [Set, Seq, Predictor]])` Retrieval-augmented generation (retrieve) new agnostic

```
(index: Index, documents: Set, query: Seq, embed: Predictor, /) -> Seq
```

SETTINGS `k: int = 4`

The k documents nearest the query's embedding, put in front of it as one prompt (then walk the LM's tree).

`HardNegatives(Search[Set, [Value, Value]])` Hard-negative mining new agnostic

```
(index: Index, queries: Value, positives: Value, /) -> Set
```

SETTINGS `k: int = 8`

The nearest elements that are not the match: harder negatives for a contrastive loss.

`KNN(Search[Prediction, [Value, Value]])` k-nearest neighbours (classification, regression) new agnostic

```
(index: Index, queries: Value, labels: Value, /) -> Prediction
```

SETTINGS `k: int = 5`

k-nearest-neighbour prediction: each query's k nearest stored examples (index.search), their labels voted (classes) or averaged (real values), weighted by distance. Nothing to fit: Insert the examples, then ask.

MODELS · model → a new model

`Wrap[M, **R = []]` — Module → Module'

```
(model: M, /, *R) -> nn.Module
```

A view of a model M: a module around it that reads its parameters (and adds none), M staying as it was. A copy with weights of its own is new state: Copy, a statement.

`Observe(Wrap[nn.Module, [nn.Module]])` Reading activations — agnostic

```
(model: nn.Module, at: nn.Module, /) -> nn.Module
```

A model that returns what one of its modules computed: the activations probes, SAEs and circuits study.

`Patch(Wrap[nn.Module, [nn.Module, Callable[[Tensor], Tensor]]])` Activation patching (circuits) — agnostic

```
(model: nn.Module, at: nn.Module, replace: Callable[[Tensor], Tensor], /) -> nn.Module
```

A model with one module's output replaced (by another input's activations): which components carry a behaviour?

`Steer(Wrap[nn.Module, [nn.Module, Tensor]])` Steering along a direction — agnostic

```
(model: nn.Module, at: nn.Module, direction: Tensor, /) -> nn.Module
```

SETTINGS `alpha: float = 1.0`

α·direction added to one module's output: features are directions, and moving along one changes the behaviour.

`Autocast(Wrap[nn.Module])` Low precision: training — agnostic

→ torch.autocast

bf16 (or fp8) compute, float32 master weights.

`Compile(Wrap[nn.Module])` Hardware-aware kernels — model-dependent

→ torch.compile

torch.compile(module) fuses its ops; attention picks flash kernels by itself.

STATEMENTS · making new state, changing it in place (!), or reading a whole dataset

CREATE · new state from a config: a model, a codec, an empty index or buffer, a copy

`MapLayer[A, B, **R = []]` builds C[a] → C[b]

`(*R) -> Model[A, B]`

Create a map: a model A → B with parameters of its own, its sizes read from examples like the data it will see.

`Network(MapLayer[Container, Prediction, [Container, Container]])` Network (each layer chosen by the types) model-dependent

`(x: Container, y: Container, /) -> Predictor`

SETTINGS `d: int = 64, depth: int = 2, mixer: AggregateLayer[Any] | None = None, mlp: MapLayer[Any, Any] | None = None`

embed → position → [aggregate, MLP] × depth → pool → readout, each slot chosen from the input and target types (EmbedFor, AggregateFor, PoolFor, ReadoutFor); mixer and mlp replace a slot's choice with a config (a scan, experts). Calling it builds the network for inputs like x and targets like y.

`Encoder(MapLayer[Container, Value, [Container]])` Embedding models (a container to one vector) model-dependent

`(x: Container, /) -> Model[Container, Value]`

SETTINGS `dim: int`

A network from any container to one vector per example: the towers of CLIP, a reward model, a critic.

`EmbedFor(MapLayer[Container, Container, [Container]])` Embeddings model-dependent

`(x: Container, /) -> Model[Container, Container]`

SETTINGS `d: int`

By what the elements hold: classes → a row of a lookup table (one more for [MASK]); real values → a linear map.

`ReadoutFor(MapLayer[Container, Container, [Container]])` Readout model-dependent

`(y: Container, /) -> Model[Container, Container]`

SETTINGS `d: int`

By what the target holds: a logit per class, a value per real number, one readout per field of a record.

`LearnedPositions(MapLayer[Container, Container, [Container]])` Positional encoding: learned model-dependent

`(x: Container, /) -> Model[Container, Container]`

SETTINGS `d: int`

A learned vector added at each position of a Seq, a Grid or a Product; a Set, a Graph or a Tree gets none.

`SinusoidalPositions(MapLayer[Container, Container, [Container]])` Positional encoding: sinusoidal model-dependent

`(x: Container, /) -> Model[Container, Container]`

SETTINGS `d: int`

Fixed sin/cos features of each position at geometrically spaced frequencies, added.

`MoE(MapLayer[Container, Container])` Mixture of experts model-dependent

`() -> Model[Container, Container]`

SETTINGS `d: int, experts: int = 8, k: int = 2`

A router sends each element to its top-k experts (MLPs) and sums their gate-weighted outputs.

`Autoencoder(MapLayer[Container, Container, [Container]])` Latent space / bottleneck model-dependent

`(x: Container, /) -> Model[Container, Container]`

SETTINGS `latent: int`

encode to a smaller container (fewer positions, few channels), decode back: the latent must keep what matters, and generation can happen there (latent diffusion). Its .encode and .decode are the two halves.

`SparseAutoencoder(MapLayer[Container, Container])` Sparse autoencoders (superposition) model-dependent

`() -> Model[Container, Container]`

SETTINGS `d: int, features: int`

Activations → a wide, sparse code → activations, trained by reconstruction plus an L1 penalty: each code unit tends to be one feature.

`LinearProbe(MapLayer[Value, Value, [Value, Value]])` Linear probes (features as directions)

`(acts: Value, labels: Value, /) -> Model[Value, Value]` model-dependent

A linear readout for frozen activations, sized by examples of them and their labels; fitted by a statement (LeastSquares, Train). Is a feature readable along one direction?

`MLP(MapLayer[Container, Container])` Linear layer + nonlinearity (MLP) model-dependent

→ nn.Linear, nn.GELU, nn.Sequential

The same small network on every element.

`SwiGLU(MapLayer[Container, Container])` Gating (SwiGLU) model-dependent

→ nn.SiLU

silu(gate(h)) · up(h), then down: one branch decides how much of the other passes.

`Normalization(MapLayer[Container, Container])` Normalization model-dependent

→ nn.RMSNorm, nn.LayerNorm

Rescale each element's features (BatchNorm aggregates over the batch instead).

`Dropout(MapLayer[Container, Container])` Dropout model-dependent

→ nn.Dropout

Zero random features while training.

`Linear(MapLayer[Value, Prediction, [Value, Container]])` Linear models (linear and logistic regression) new model-dependent

`(x: Value, y: Container, /) -> Model[Value, Prediction]`

SETTINGS `bias: bool = True`

A linear model Wx + b on feature vectors, sized by examples of its inputs and targets: linear regression when the target is real values (Loss picks squared error), logistic or softmax regression when it is classes (cross-entropy), a linear SVM with Hinge. Fitted by Train (a gradient), or LeastSquares (closed form).

`KernelFeatures(MapLayer[Value, Value, [Value]])` Kernel methods (random features) new

`(x: Value, /) -> Model[Value, Value]` model-dependent

SETTINGS `features: int = 1024, bandwidth: float = 1.0`

Random Fourier features of a shift-invariant kernel (RBF by default): a Linear on them is a kernel machine, so kernel ridge regression is LeastSquares on it and a kernel SVM is Hinge on it.

`Residual(MapLayer[Container, Container, [nn.Module]])` Residual connections model-dependent

`(f: nn.Module, /) -> Model[Container, Container]`

h + f(norm h): the layer edits the stream instead of replacing it.

`AggregateLayer[C, In = C, **R = []]` builds C[h] → C[h]

`(*R) -> Model[In, C]`

Create an aggregate: a model whose elements read the ones they are related to, a C out (reading a C, unless In says what else it reads: a memory, a second container).

`AggregateFor(AggregateLayer[Container, Container, [Container]])` Choosing the aggregation from the container — model-dependent

`(x: Container, /) -> Model[Container, Container]`

SETTINGS `d: int`

What each element reads, from the container: Graph, Tree → message passing · Product → axial · causal Seq → causal attention (a selective scan when long) · large Grid → convolution · else attention over all.

`Attention(AggregateLayer[Container])` Attention (self, causal) — model-dependent

`() -> Model[Container, Container]`

SETTINGS `d: int, causal: bool = False`

Each element reads every other (or every earlier one), weighted by query·key similarity.

`CrossAttention(AggregateLayer[Container, tuple[Container, Container]])` Cross-attention — model-dependent

`() -> Model[tuple[Container, Container], Container]`

SETTINGS `d: int`

Queries from one container, keys and values from another (text reading an image).

`Conv(AggregateLayer[Grid])` Convolution — model-dependent

`() -> Model[Grid, Grid]`

SETTINGS `d: int, size: int = 3`

Each element reads its neighbours on the lattice, with the same weights at every offset.

`Recurrence(AggregateLayer[Seq])` Recurrence (RNN / LSTM / GRU) new — model-dependent

`() -> Model[Seq, Seq]`

SETTINGS `d: int`

STATE a hidden vector, carried along the sequence

Each element reads a state carried from the earlier ones, updated by a gated nonlinear function.

`SelectiveScan(AggregateLayer[Seq])` State-space models / linear recurrences new — model-dependent

`() -> Model[Seq, Seq]`

SETTINGS `d: int`

STATE s_t = a_t·s_{t−1} + b_t, run as a parallel scan

A linear recurrence whose coefficients come from the input (S4, Mamba, linear attention): every earlier element read, in linear time.

`MessagePassing(AggregateLayer[Graph])` Message passing (GNN) — model-dependent

`() -> Model[Graph, Graph]`

SETTINGS `d: int`

Each node reads its own features and the mean along its incoming edges; in a tree, children pass up to parents.

`Axial(AggregateLayer[Product])` Axial (products of structures) — model-dependent

`() -> Model[Product, Product]`

SETTINGS `d: int, parts: tuple[str, ...] = ('Seq', 'Grid')`

One aggregation per factor of a product (frames, then pixels), the other factors as batch; summed.

`UNet(AggregateLayer[Grid])` Encoder–decoder hierarchy (U-Net) model-dependent

`() -> Model[Grid, Grid]`

SETTINGS `d: int, levels: int = 2`

Down to a coarse grid and back up, with skip connections: each element reads a wide neighbourhood.

`Retrieval(AggregateLayer[Container, tuple[Container, Index]])` Retrieval (attention over a memory) model-dependent

`() -> Model[tuple[Container, Index], Container]`

SETTINGS `k: int`

Each element reads the k nearest entries of an external memory (an Index), softmax-weighted.

`FoldLayer[C, A, **R = []]`

builds C[a] → a

`(*R) -> Model[C, A]`

Create a fold: a model that reduces a C to an A.

`PoolFor(FoldLayer[Container, Value, [Container]])` Pooling (mean / last position) model-dependent

`(x: Container, /) -> Model[Container, Value]`

A container's features → one vector per example: a causal Seq keeps its last position, the one that read everything; anything else, the mean.

`Encode[A, B, **R = []]`

builds A ⇄ C[classes]

`(*R) -> Codec[A, B]`

Create a codec: raw data A written in a vocabulary B, and read back. One that learns is fitted by FitCodec.

`Tokenizer(Encode[list[str], Seq])` Tokenization (BPE, SentencePiece, bytes) new model-dependent

`() -> Codec[list[str], Seq]`

SETTINGS `kind: str = "bpe", vocab: int = 32000`

Text ⇄ a sequence of ids, built empty: BPE merges, fitted to a corpus by FitCodec (bytes alone: a fixed vocabulary of 256, nothing to fit). The tokenizer decides the content type: Seq of classes(vocab).

`ImageTokenizer(Encode[Grid, Grid, [Grid]])` Image tokenizer (VQ-VAE, VQGAN) new model-dependent

`(images: Grid, /) -> Codec[Grid, Grid]`

SETTINGS `codes: int = 1024`

An image ⇄ a coarser grid of codebook ids: an encoder, vector quantization, a decoder, trained to reconstruct.

`AudioCodec(Encode[Seq, Product, [Seq]])` Audio codec (EnCodec, residual quantization) new model-dependent

`(audio: Seq, /) -> Codec[Seq, Product]`

SETTINGS `codebooks: int = 8, codes: int = 1024`

A waveform ⇄ frames × codebooks of ids: each codebook quantizes what the previous ones left (residual VQ).

`Empty[S, **R = []]` ... → Index | Buffer

`(*R) -> S`

Create an empty store S, to fill with statements: an index (its sizes from R), a buffer.

`EmptyIndex(Empty[Index, [Value]])` Index (an empty one) **new** agnostic

`(like: Value, /) -> Index`

SETTINGS `kind: str = "flat"`

An empty index for vectors like these (their size): flat (exact), IVF (clusters), HNSW (a neighbour graph), PQ (compressed codes), LSH (hash buckets), inverted (BM25), sum tree (by priority). Statements train and fill it.

`ReplayBuffer(Empty[Buffer[Any]])` Replay buffer (empty) **new** agnostic

`() -> Buffer[Any]`

SETTINGS `size: int, priority: bool = False`

An empty buffer of at most size examples, read by sampling (uniformly, or by priority); Extend fills it.

`Copy[M]` Module → Module

`(x: M, /) -> M`

Create a copy of M with weights of its own, as M is now: a snapshot that later changes to M do not reach.

`FrozenCopy(Copy[nn.Module])` Frozen copy (reference / target network) agnostic

`[M: Module](model: M, /) -> M`

A copy that no longer trains: DPO's and RLHF's reference, DQN's target network.

`Sklearn[E: Estimator]` Estimator → SkModel

a scikit-learn estimator, as a model: fitted by a statement, then called on batches

`Sklearn` scikit-learn estimator **new** agnostic

`() -> SkModel[E]`

SETTINGS `estimator: E`

Create a classical model from a scikit-learn estimator, or anything with its API (xgboost, lightgbm, umap): a fresh copy each call, so each is a model of its own. FitEstimator fits it; then its predict, predict_proba, transform, ... are expressions on batches.

`BoostedTrees(Sklearn[Estimator])` Decision trees, random forests, gradient boosting **new** agnostic

→ sklearn.ensemble.HistGradientBoostingClassifier / Regressor, RandomForestClassifier, sklearn.tree.DecisionTreeClassifier, xgboost, lightgbm

SETTINGS `estimator: E`

Decision trees on feature vectors: one (CART), many on bootstrap samples averaged (a random forest), or each fitted to the errors of the ones before it (gradient boosting). Classifier or regressor, by the estimator.

`SupportVectorMachines(Sklearn[Estimator])` Support vector machines (kernel SVM) **new** agnostic

→ sklearn.svm.SVC, SVR

SETTINGS `estimator: E`

The widest-margin separator in a kernel's feature space (a linear one is Linear with Hinge).

`NaiveBayes(Sklearn[Estimator])` Naive Bayes **new** agnostic

→ sklearn.naive_bayes.GaussianNB, MultinomialNB

SETTINGS `estimator: E`

A generative classifier whose features are independent given the class: counts or one Gaussian per feature.

`GaussianProcesses(Sklearn[Estimator])` Gaussian processes `new` agnostic
→ sklearn.gaussian_process.GaussianProcessRegressor, GaussianProcessClassifier
SETTINGS `estimator: E`
A distribution over functions, set by a kernel: a prediction and its uncertainty, from every training example.

`KMeansClustering(Sklearn[Estimator])` k-means `new` agnostic
→ sklearn.cluster.KMeans, MiniBatchKMeans
SETTINGS `estimator: E`
k centroids, each example to its nearest and each centroid to the mean of its examples, until nothing moves; predict gives a new example's cluster.

`GaussianMixtureModel(Sklearn[Estimator])` Mixture models (GMM, EM) `new` agnostic
→ sklearn.mixture.GaussianMixture, BayesianGaussianMixture
SETTINGS `estimator: E`
k weighted Gaussians fitted by expectation–maximization: a density, and soft clusters by responsibility.

`PrincipalComponents(Sklearn[Estimator])` PCA (dimensionality reduction) `new` agnostic
→ sklearn.decomposition.PCA, IncrementalPCA, TruncatedSVD
SETTINGS `estimator: E`
A linear map onto the directions of most variance; transform projects onto them.

`Standardization(Sklearn[Estimator])` Standardization (feature scaling) `new` agnostic
→ sklearn.preprocessing.StandardScaler, MinMaxScaler, RobustScaler
SETTINGS `estimator: E`
Each feature shifted and scaled (to mean 0 and variance 1, or into a range), by statistics of the training data only, so that the validation and test data are scaled as it was.

UPDATE · a model, a codec, an index or a buffer, changed in place

`Fit[X, **R = []]` X! × Dataset

```
(x: X, /, *R) -> None
```

Fit an X to data, in place: a model by a loss's gradient or a closed form, a codec or an estimator by its own rule.

`Train(Fit[nn.Module, [Callable[..., Tensor], Dataset[Any] | None]])` Training `new` agnostic
`(model: nn.Module, objective: Callable[..., Tensor], data: Dataset[Any] | None, /) -> None`
SETTINGS `steps: int, optimizer: Scan[Tensor, Any, Tensor] = Optimizer(kind='adamw', lr=0.0003), schedule: Scan[int, int, float] | None = None, max_grad_norm: float | None = 1.0, precision: torch.dtype | None = None`
Descend the objective over the dataset's batches: each step, objective(model, *batch) (under autocast to precision), its gradient (clipped to max_grad_norm), the optimizer's update of the parameters, in place, at the schedule's rate. In training mode only here, so a forward's own effects (BatchNorm statistics) are training's too. With no data, the objective makes its own each step (it walks the model: RL, self-play).

`FitCodec(Fit[Codec[Any, Any], [Dataset[Any]]])` Codec fitting (BPE merges, a vocabulary) `new` agnostic
`(codec: Codec[Any, Any], data: Dataset[Any], /) -> None`
Fit a codec to a corpus by its own rule, in place: BPE merges the most frequent pairs until the vocabulary is full, a unigram model prunes. (A learned codec, VQ-VAE or EnCodec, is a model: train.)

`LeastSquares(Fit[nn.Module, [Dataset[Any]]])` Least squares (closed form, ridge) **new** model-dependent

```
(model: nn.Module, data: Dataset[Any], /) -> None
```

SETTINGS `ridge: float = 0.0`

Solve for a Linear model's weights in closed form, in place: $(X^TX + \lambda I)^{-1}X^Ty$, X^TX and X^Ty accumulated over the dataset's batches; ridge regression when ridge is above 0.

`FitEstimator(Fit[SkModel[Any], [Dataset[Any]]])` Fitting an estimator (fit, partial_fit) **new** agnostic

```
(model: SkModel[Any], data: Dataset[Any], /) -> None
```

SETTINGS `incremental: bool | None = None`

Fit an estimator's model to a dataset, in place: batch by batch by partial_fit when incremental (by default, whenever the estimator has it), otherwise every batch gathered and fit once. Each batch is (features, target), or features alone for an unsupervised one.

`TrainingStep(Fit[nn.Module])` Training step (clipping, mixed precision) agnostic

→ Train

Train's settings: autocast the objective, clip the gradient norm.

`TwoPlayerGames(Fit[nn.Module])` Two-player games (alternating fits) agnostic

→ repeat, Train

No single quantity goes down: each player's Train on its own loss, in turn, inside a repeat (GANs, adversarial training).

`Assign[X, **R = []]` Weights!

```
(x: X, /, *R) -> None
```

Set an X's parameter values directly, without a gradient: copied in, initialized, averaged.

`Transfer(Assign[Weights, [Weights]])` Pretrain → finetune (transfer) agnostic

```
(target: Weights, source: Weights, /) -> None
```

Start a model from a trained one: copy into the target the source's weights whose names and shapes match (the body), keep the target's head.

`Init(Assign[Weights])` Initialization agnostic

```
(model: Weights, /) -> None
```

Fan-in normal for matrices, zeros for output projections: each residual block starts as the identity.

`EMA(Assign[Weights, [Weights]])` EMA of weights agnostic

```
(average: Weights, model: Weights, /) -> None
```

SETTINGS `decay: float = 0.999`

average ← decay·average + (1 − decay)·model, in place: after each round or step, a smoother copy, and the slow target of self-distillation and DQN.

`Alter[X, **R = []]` Module!

```
(x: X, /, *R) -> None
```

Change what an X's parameters are: add some, freeze some, quantize, shard.

`Freeze(Alter[nn.Module, [Callable[[str], bool]]])` Freezing agnostic

```
(model: nn.Module, trainable: Callable[[str], bool], /) -> None
```

Train only the parameters whose name passes a test.

`LoRA(Alter[nn.Module])` LoRA — model-dependent

```
(model: nn.Module, /) -> None
SETTINGS rank: int = 8
```

W + B·A on chosen linear layers, the base frozen: the finetuning update, kept low-rank.

`Adapter(Alter[nn.Module])` Adapters — model-dependent

```
(model: nn.Module, /) -> None
SETTINGS bottleneck: int = 8
```

After chosen frozen layers, a small bottleneck MLP that trains alone, starting at zero.

`PromptTuning(Alter[nn.Module])` Prompt tuning — model-dependent

```
(model: nn.Module, /) -> None
SETTINGS n: int
```

Learned vectors in front of a frozen model's input; only they train.

`Quantize(Alter[Weights])` Quantization (int8 / int4) — model-dependent

```
(model: Weights, /) -> None
SETTINGS bits: int = 8
```

Weights rounded to 2^(bits−1) − 1 levels a side per row: computing as the low-precision model would.

`LowPrecisionInference(Alter[Weights])` Low precision: inference — model-dependent

→ Quantize

int8 / int4 weights.

`Parallelism(Alter[nn.Module])` Parallelism — agnostic

→ torch.distributed

Data, sharded, tensor, pipeline, expert, sequence: fully_shard shards a module in place; DTensor places its weights.

`Store[X, **R = []]` — Index! / Buffer!

```
(x: X, /, *R) -> None
```

Keep a store X up to date: learn an index's structure, insert into it, delete from it, add to a buffer.

`TrainIndex(Store[Index, [Value]])` Index training (IVF centroids, PQ codebooks) new — agnostic

```
(index: Index, sample: Value, /) -> None
```

Learn the index's structure from a sample of the vectors, in place: k-means centroids for IVF, codebooks for PQ (flat and HNSW have nothing to learn). CREATE INDEX, before any rows.

`Insert(Store[Index, [Dataset[Set]]])` Index insertion new — agnostic

```
(index: Index, data: Dataset[Set], /) -> None
```

Add every batch of the dataset to the index, in place (FAISS add, INSERT). An index over an expression's output (embed(x)) goes stale when that expression's model is trained again: build it again.

`Delete(Store[Index, [Tensor]])` Index deletion new — agnostic

```
(index: Index, ids: Tensor, /) -> None
```

Remove elements from the index by id, in place (FAISS remove_ids, DELETE).

`Extend(Store[Buffer[Container], [Container]])` Continual learning (replay) — agnostic

```
[C: Container](buffer: Buffer[C], batch: C, /) -> None
```

Add a batch to a bounded buffer, in place, dropping the oldest (or the lowest priority): past experience to train on again (continual learning, off-policy RL, prioritized replay).

DATASET · read every batch, change nothing

`Source[D, **R = []]` … → Dataset

```
(*R) -> D
```

Where batches come from, a dataset D: memory, an environment, a buffer, an expression on each batch.

`Batches(Source[Indexed[Any], ...])` Minibatches **new** agnostic

```
(*data: Container) -> Indexed[Any]
```

SETTINGS `size: int, shuffle: bool = True`

Containers in memory as a dataset: random minibatches of size examples, the containers sliced together (a tuple per batch when there are several). Indexed, so it can be curated and split.

`Collector(Source[Dataset[Path], [Environment, Predictor]])` Collector (walks of a live policy) **new** agnostic

```
(env: Environment, policy: Predictor, /) -> Dataset[Path]
```

SETTINGS `steps: int, batch: int`

Walks of the environment with the policy as it is when each batch is drawn: on-policy data, read inside the fit that changes the policy (torchrl's collectors).

`Mix(Source[Indexed[Any], ...])` Data mixture weights agnostic

```
(*datasets: Indexed[Any]) -> Indexed[Any]
```

SETTINGS `weights: list[float]`

Several datasets as one (web, code, books), each example weighted so that batches draw from them in proportion to the weights: the mixture as one Indexed, to curate and split like any.

`Each(Source[Dataset[Any], [Callable[..., Any], Dataset[Any]]])` An expression on each batch **new** agnostic

```
(f: Callable[..., Any], data: Dataset[Any], /) -> Dataset[Any]
```

f applied to every batch, once per batch as the dataset batches it (never re-batched, since f may depend on what a batch holds): how a statement reads an expression's output, embeddings to index, features to probe.

`Curate[C, **R = []]` Indexed → Subset

```
(data: Indexed[C], /, *R) -> Subset[C]
indices(data: Indexed[C], /, *R) -> Tensor
```

Keep, drop, reorder or resample examples, looking at all of them (blocking): a sub-dataset of the one given. A method only chooses the indices; the group makes them a Subset, which batches, iterates and logs them.

`Dedup(Curate[Any])` Data deduplication agnostic

```
indices(data: Indexed[Any], /) -> Tensor
```

Drop exact copies across the whole dataset (by hash): the first of each kept.

`DropNearDuplicates(Curate[Any])` Near-duplicate removal (MinHash / LSH) **new** agnostic

```
indices(data: Indexed[Any], /) -> Tensor
```

SETTINGS `threshold: float = 0.8`

Drop examples whose MinHash signatures collide with an earlier one's above a similarity threshold: dedup beyond exact copies.

`KeepBest(Curate[Any, [Callable[[Any], Value]]])` Data filtering by fraction **new** agnostic

```
indices(data: Indexed[Any], score: Callable[[Any], Value], /) -> Tensor
```

SETTINGS `keep: float = 0.5`

The best-scoring fraction of the whole dataset: every score first, then the ones above their quantile.

`Curriculum(Curate[Any, [Callable[[Any], Value]]])` Curriculum learning agnostic

`indices(data: Indexed[Any], difficulty: Callable[[Any], Value], /) -> Tensor`

The dataset sorted by difficulty, easy first: a global sort, so it reads every batch before the first comes out.

`Subsample(Curate[Any])` Random subsampling new agnostic

`indices(data: Indexed[Any], /) -> Tensor`

SETTINGS `fraction: float, seed: int = 0`

A random fraction of the examples, without replacement: a smaller run, a quick look.

`Resample(Curate[Any, [Callable[[Any], Value]]])` Resampling (importance sampling, bootstrap) new agnostic

`indices(data: Indexed[Any], weight: Callable[[Any], Value], /) -> Tensor`

SETTINGS `n: int | None = None, seed: int = 0`

n examples drawn with replacement, each in proportion to its weight: importance sampling, or the bootstrap when every weight is 1 (n defaults to the dataset's size).

`Rebalance(Curate[Any, [Callable[[Any], Tensor]]])` Class rebalancing (over- and undersampling) new agnostic

`indices(data: Indexed[Any], label: Callable[[Any], Tensor], /) -> Tensor`

SETTINGS `mode: str = "oversample", seed: int = 0`

As many examples of each class as of the largest ("oversample", repeating the rarer ones) or the smallest ("undersample", dropping some of the commoner ones).

`Partition[C, **R = []]` Indexed → splits of Subsets

`(data: Indexed[C], /, *R) -> tuple[tuple[Subset[C], ...], ...]`

`parts(data: Indexed[C], /, *R) -> tuple[tuple[Tensor, ...], ...]`

Split a dataset into sub-datasets: a list of splits, each a tuple of parts (train and test, or train, validation and test). A method only chooses the indices of each part; the group makes them Subsets.

`Holdout(Partition[Any])` Train / validation / test split new agnostic

`parts(data: Indexed[Any], /) -> tuple[tuple[Tensor, ...], ...]`

SETTINGS `fractions: tuple[float, ...] = (0.8, 0.1, 0.1), label: Callable[[Any], Tensor] | None = None, seed: int = 0`

One split, at random, into parts of the given fractions (train, validation, test), each class in the same proportions in every part when label says what an example's class is (stratified).

`Folds(Partition[Any])` Splits from a splitter new agnostic

`parts(data: Indexed[Any], /) -> tuple[tuple[Tensor, ...], ...]`

SETTINGS `splitter: Splitter, label: Callable[[Any], Tensor] | None = None, groups: Callable[[Any], Tensor] | None = None`

The splits of any scikit-learn splitter, each a (train, test) pair: label gives it the classes to stratify by, groups the groups to keep together (each called on batches, one value per example).

`CrossValidation(Folds)` k-fold cross-validation new agnostic

→ sklearn.model_selection.KFold, StratifiedKFold, GroupKFold, TimeSeriesSplit

SETTINGS `splitter: Splitter, label: Callable[[Any], Tensor] | None = None, groups: Callable[[Any], Tensor] | None = None`

k (train, test) splits, each example tested once: plain, stratified by class, by group, or forward in time.

`Reduce[C, A, **R = []]` Dataset → a

`(data: Dataset[C], /, *R) -> A`

One value A from a whole dataset of C: a metric over every batch, a statistic, one batch.

`Evaluate(Reduce[Any, Tensor, [Callable[..., Tensor]]])` Evaluation (a metric over a dataset) **new**

agnostic

`(data: Dataset[Any], metric: Callable[..., Tensor], model: nn.Module | None = None, /) -> Tensor`

A per-batch metric (accuracy, a loss, perplexity), averaged over every batch, weighted by their sizes; given a model, the metric is called as metric(model, *batch), as a loss is.

`Metric(Reduce[tuple[Tensor, Tensor], Any])` Whole-dataset metric **new**

agnostic

`(data: Dataset[tuple[Tensor, Tensor]], /) -> Any`

SETTINGS `fn: Callable[[Any, Any], Any]`

A metric of every prediction and target at once, not a mean of per-batch values: data is (prediction, target) pairs, gathered, then fn(targets, predictions), as scikit-learn's metrics take them.

`ClassificationMetrics(Metric)` Classification metrics (AUC, F1, precision / recall, confusion matrix) **new**

→ sklearn.metrics.roc_auc_score, f1_score, precision_recall_fscore_support, confusion_matrix

agnostic

SETTINGS `fn: Callable[[Any, Any], Any]`

The ones that need every score ranked or counted together: ROC AUC, F1, precision and recall, a confusion matrix.

`Streaming(Reduce[tuple[Tensor, Tensor], Tensor])` Streaming metric **new**

agnostic

`(data: Dataset[tuple[Tensor, Tensor]], /) -> Tensor`

SETTINGS `metric: StreamMetric`

The same number as Metric without keeping every prediction: the metric is reset, updated on each (prediction, target) batch, then computed (torchmetrics' API).

`StreamingMetrics(Streaming)` Calibration error, FID (streaming) **new**

agnostic

→ torchmetrics.classification.CalibrationError, torchmetrics.image.fid.FrechetInceptionDistance

SETTINGS `metric: StreamMetric`

Metrics of a whole dataset that keep running totals instead of every prediction: calibration error, FID.

`FitPredict(Reduce[Any, Tensor])` Fit and predict (transductive) **new**

agnostic

`(data: Dataset[Any], /) -> Tensor`

SETTINGS `estimator: Estimator`

A fresh clone of the estimator fitted to the whole dataset, and what it says of each example: for methods that cannot label new data, only the data they were fitted to (fit_predict).

`DensityClustering(FitPredict)` Density clustering (DBSCAN, HDBSCAN) **new**

agnostic

→ sklearn.cluster.DBSCAN, HDBSCAN, OPTICS

SETTINGS `estimator: Estimator`

Clusters as dense regions, how many found rather than given, some examples left as noise.

`HierarchicalClustering(FitPredict)` Hierarchical and spectral clustering **new**

agnostic

→ sklearn.cluster.AgglomerativeClustering, SpectralClustering

SETTINGS `estimator: Estimator`

Clusters merged bottom up, the closest pair first, or cut from a graph of similarities (spectral).

`FitTransform(Reduce[Any, Tensor])` Fit and transform (transductive) **new** agnostic

`(data: Dataset[Any], /) -> Tensor`

SETTINGS `estimator: Estimator`

A fresh clone of the estimator fitted to the whole dataset, and each example's new features: for embeddings that place only the data they were fitted to (fit_transform).

`Embedding2D(FitTransform)` Neighbour embeddings (t-SNE, UMAP) **new** agnostic

→ sklearn.manifold.TSNE, umap.UMAP

SETTINGS `estimator: Estimator`

A low-dimensional map that keeps each example's neighbours near it: for looking at data, not for features.

`Accumulate(Reduce[Any, Any, [Scan[Any, Any, Any]]])` Accumulate (a scan across batches) **new**

`(data: Dataset[Any], s: Scan[Any, Any, Any], /) -> Any` agnostic

A Scan's state carried across a dataset's batches, its final state returned: normalization statistics, a histogram, any metric that is not a mean.

`Peek(Reduce[Any, Any])` First batch **new** agnostic

`[C](data: Dataset[C], /) -> C`

The first batch of a dataset: what an expression can see of it, its type, to build a network or an index for.

Intents · abstract operators left open (resolve.py)

An intent is written where a method could be and printed with a `?`. A resolver picks one of its candidates (the default, a given choice, or by cost) and one rewrite lowers the program to concrete methods.

> **`AugmentImage(Intent, Convert[Grid, Grid])`**
> CANDIDATES `RandomFlip(), RandomCrop(), ColorJitter(), GaussianNoise()`
> Some label-preserving random change of an image: a flip, a crop, colour jitter, noise.

> **`Decode(Intent, Choose)`**
> CANDIDATES `Greedy(), Sample(), Beam(k=4)`
> Some way to walk an LM's tree: the likeliest token, a sample, a beam.

> **`FillOrder(Intent, Order)`**
> CANDIDATES `Together(), MostConfident(), InOrder(), PerElement()`
> Some order to fill holes in: all together (diffusion), the surest first, left to right, each its own level.

> **`Mixer(Intent, AggregateLayer[Seq])`**
> CANDIDATES `Attention(d=64, causal=True), SelectiveScan(d=64), Recurrence(d=64)`
> Some way for a sequence's elements to read the earlier ones: attention, a selective scan, a recurrence.

> **`TrainLength(Intent, Fit[nn.Module, [Callable[..., Tensor], Dataset[Any] | None]])`**
> CANDIDATES `Train(steps=100), Train(steps=200), Train(steps=400)`
> Some amount of training: how long to update the network.

Meta-primitives and phenomena

Meta-primitives · properties of the groups

- **Bake in symmetry by sharing weights** aggregate
  the same weights wherever the container relates elements: conv on a grid, attention on a set, messages on a graph
- **Let the input decide the computation** aggregate, map
  attention weights, scan coefficients and expert choices computed from the data
- **Predict the hidden part from the visible part** split, fill
  every self-supervised signal is a split into Holes; generation fills them back
- **Turn one hard step into many easy ones** fill, walk, unroll
  a token, a denoising step, an update at a time: a state, stepped
- **Learning is compression** map, split
  bottlenecks, codebooks and students: the target, from a smaller description
- **Bootstrap from yourself** split
  targets made by your own (older) model: TD, self-distillation, self-play, AlphaZero
- **Amortize, or search** map, walk, reshape, search
  one forward pass that answers at once, or a tree grown by search
- **Every loss is a likelihood** score
  Loss()(pred, target): what the target holds picks which
- **Not every objective is a loss** fold, score
  a return is a fold of a path; its gradient comes from scoring the path, or there is none
- **Structure decides the computation** aggregate
  the container picks what each element reads

Phenomena · observed, not computed

- **Implicit regularization of SGD**: a property of training dynamics, seen in eval curves
- **Double descent / benign overfitting**: seen by sweeping model or data size
- **Grokking**: delayed generalization in long runs
- **Superposition**: more features than dimensions; measured with sparse autoencoders
- **General methods + scale beat hand-engineering**: an empirical lesson (the bitter lesson), not an op